Drew University - CSCI 260 - Fall 2024
In-Class Exercise 4 - C Warm-ups

Extra Credits: If you submit your programs in a .txt file on Moodle by 11/11 (M) 11:59pm, you will be able to earn extra credits that make up the points lost in HWs. 11/06/2024

Implement the following C programs in Linux's *nano* text editor on CoCalc.
To compile your C source file, type *gcc <filename.c>*; and to run the file just built, type *./a.out*.

1. **Printing Triangles**: Given value of an *int* variable n, print out a triangle made of *s using *for-loops*.
   Ex.: If $n = 4$, then after the program is executed, the console should contain:
   ```
   *
   **
   ***
   ****
   ```

2. **The Fibonacci Sequence**: The Fibonacci Sequence $F(n)$ is defined as $F(0) = F(1) = 1$ and $F(n) = F(n-1) + F(n-2)$ for any n at least 2. Given value of an *int* variable $n \geq 0$, print out all the $F(n)$ values between 0 and n.
   Ex.: If $n = 7$, then after the program is executed, the console should contain: "1, 1, 2, 3, 5, 8, 13, 21".

3. (1 point) **Transpose of a Matrix**:A matrix M is a 2-dimensional array. We can access the array entries by providing the index values of the row and column (both starting with the 0th. index); e.g., for an *int* array M, $M[2][3] = 5$ means the the value contained in row 2 and column 3 is 5.
   A transpose M^T of M is defined as follow: for all the indices i, j, $M^T[i][j] = M[j][i]$.
   Given an *int* matrix M, obtain M^T and print out its entries.
   Ex.: If *int* $M[3][5] = \{\{1,2,3,4,5\},\{5,4,3,2,1\},\{0,0,0,0,0\}\}$, then its transpose is *int* $MT[5][3] = \{\{1,5,0\},\{2,4,0\},\{3,3,0\},\{4,2,0\},\{5,1,0\}\}$.
   Note: To get the dimension of an array, we can use the *sizeof()* function. However, if you simply use *sizeof(M)*, a value of 60 (bytes) is returned. If *sizeof(M[0])* is used, then 20 (bytes) is returned. Lastly, *sizeof(M[0])/sizeof(int)* gives you the number of columns. Then, we can compute the number of rows.

4. (2 point) **Printing Upside-down Triangle**: Similar to the triangle problem above, but the triangle is now upside-down and you are asked to use *while-loops* instead.
   Ex.: If $n = 4$, then after the program is executed, the console should contain:
   ```
   ****
   ***
   **
   *
   ```

5. (2 point) **Prime Numbers**: Print out all the prime numbers between 2 and an *int* variable n.
   Ex.: If $n = 10$, then after the program is executed, the console should contain: "2, 3, 5, 7".